

Architecture, base de données et interface

Une boutique de mangas en ligne : comment le projet est rangé, ce que la base contient, comment l'administration l'écrit, et ce qui rend les pages vivantes.

PHP

PDO

MYSQL

JAVASCRIPT

HTML/CSS

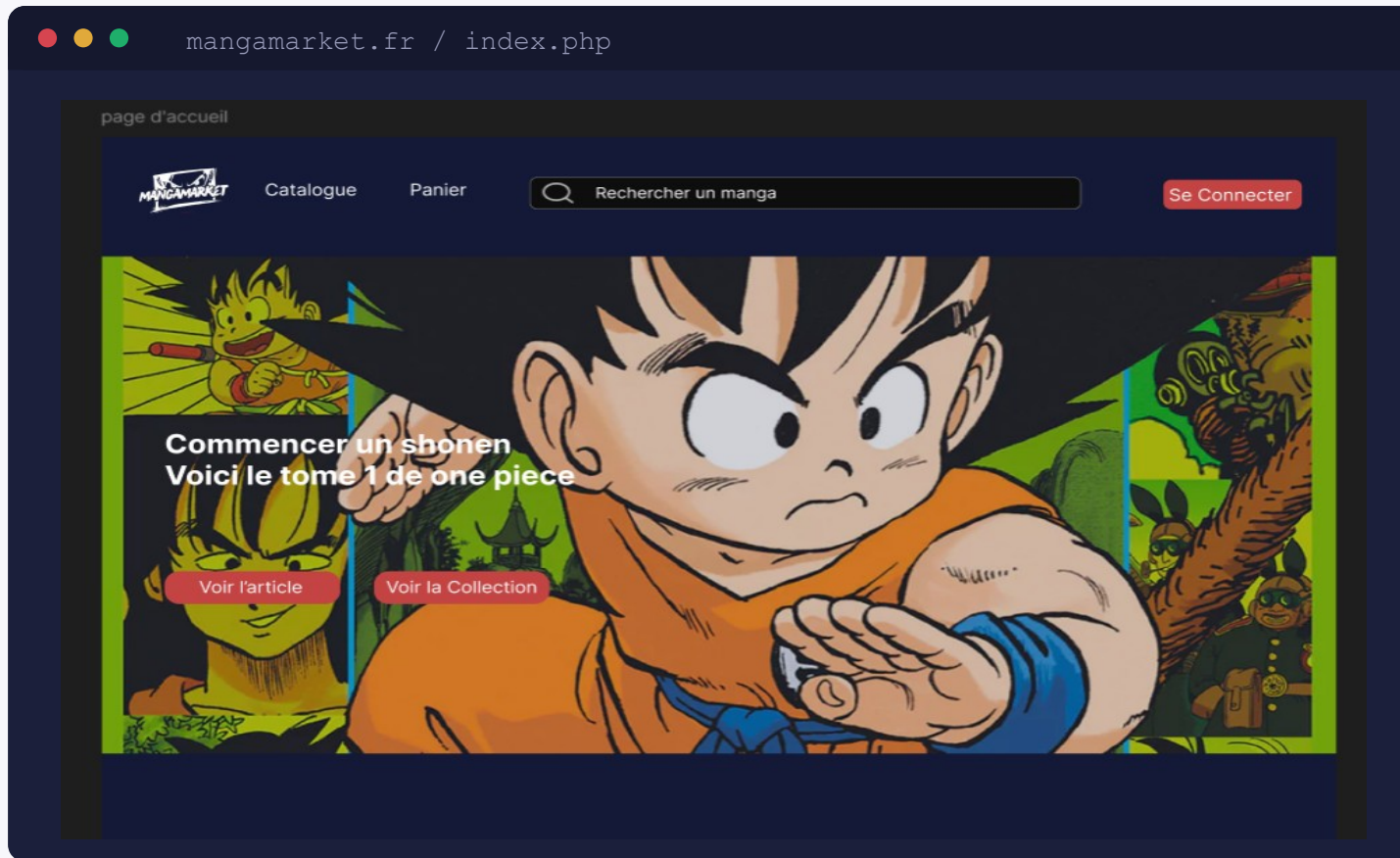
```
catalogue.php

1 GET catalogue.php?categories[]=shonen
2   → $catsActives = ['shonen']; $prixMax =
10.0
3   → $where[] = "c.slug IN (:cat0)"
4   → $pdo->prepare($sql)->execute($params)
5   ← 14 produits → rendu HTML direct
```

Chaque capture de ce diaporama est un extrait réel du dépôt du projet.

Le site dont on parle

Un catalogue de mangas vendus tome par tome, avec panier, commande, compte client et back-office administrateur.



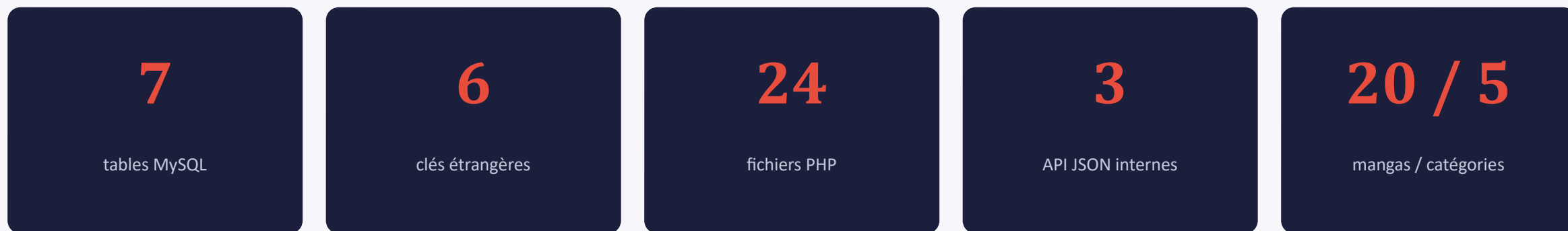
UNE PAGE, PLUSIEURS RÔLES

- L'accueil affiche un produit vedette et une sélection « tendances » tirée aléatoirement en base
- Le header gère trois états : invité, client connecté, administrateur — chacun avec son propre menu
- Rien n'est statique : catégories, prix et stock viennent tous de MySQL à chaque chargement

INTRODUCTION

MangaMarket en chiffres

Les ordres de grandeur du projet, mesurés sur le dépôt.



PILE TECHNIQUE



Aucun framework front ou back : choix assumé pour démontrer la maîtrise des fondamentaux exigés par le référentiel du titre [DWWM](#).

Le plan de la soutenance

BLOC A

L'architecture

Arborescence du projet et trajet d'une requête

BLOC B

Le front-end

Maquettage, intégration HTML/CSS, JavaScript

BLOC C

Le back-end & les données

Base de données, accès aux données, transactions

BLOC D

Sécurité & démonstration

Mesures de sécurité et parcours utilisateur

BLOC E

Bilan

Compétences mobilisées, difficultés, conclusion

BLOC A

L'architecture

Comment le projet est rangé, et par où passe une requête de bout en bout.

Arborescence

Trajet d'une requête

Une arborescence en quatre rôles

Vues, API, fichiers partagés et assets : chaque dossier a une responsabilité unique.

```
mangamarket/ — racine du dépôt

1 mangamarket/
2 |— index.php          → accueil
3 |— views/             → 10 pages client
4 |   |— catalogue.php · produit.php
5 |   |— panier.php · achat.php
6 |   |— admin/         → 5 pages back-office
7 |— api/               → 3 endpoints JSON
8 |   |— panier · produits · search_api
9 |— includes/          → 5 fichiers partagés
10 |   |— db · header · footer · auth_*
11 |— asset/css/ · asset/js/ → 10 + 10
12 |— mangamarket.sql    → 7 tables
```

CE QUE CETTE SÉPARATION GARANTIT

- includes/db.php est le seul point de connexion PDO : toutes les pages le réutilisent, aucune n'ouvre sa propre connexion
- api/ ne rend jamais de HTML : uniquement du JSON, consommé en Fetch par le front
- views/admin/ est protégée par auth_admin.php, inclus en première ligne de chaque page

Un header, trois rôles possibles

includes/header.php est inclus par toutes les pages : il décide seul de ce qui s'affiche.

```
includes/header.php

1  $connected  = !empty($_SESSION['user_id']);
2  $userRole   = $_SESSION['user_role'] ?? 'client';
3  $isAdmin    = $connected && $userRole === 'admin';
4
5  <?php if ($isAdmin): ?>
6      <a href="admin/dashboard.php">Dashboard</a>
7  <?php elseif ($connected): ?>
8      <a href="profil.php">Mon profil</a>
9  <?php else: ?>
10     <a href="creecompte.php">Se connecter</a>
```

UN SEUL FICHIER, TROIS MENUS

- Invité : lien « Se connecter », panier localStorage
- Client connecté : lien profil, panier BDD, badge de commandes
- Administrateur : icône , menu dashboard / commandes / produits / utilisateurs / messages
- Le panier est même masqué pour les admins : ils n'ont pas besoin d'acheter

Une connexion PDO, configurée une seule fois

includes/db.php centralise les options de sécurité et de confort de toute l'application.

```
includes/db.php

1 $dsn = sprintf('mysql:host=%s;dbname=%s;charset=%s',
2   DB_HOST, DB_NAME, DB_CHARSET);
3
4 $options = [
5   PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
6   PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
7   PDO::ATTR_EMULATE_PREPARES => false,
8 ];
9
10 $pdo = new PDO($dsn, DB_USER, DB_PASS, $options);
```

TROIS DÉCISIONS, UN SEUL ENDROIT

- `ERRMODE_EXCEPTION` : une erreur SQL lève une exception au lieu d'échouer silencieusement
- `FETCH_ASSOC` : chaque ligne devient un tableau associatif — pas d'index numériques à retenir
- `EMULATE_PREPARES` à `false` : MySQL exécute de vraies requêtes préparées, pas une simulation PHP
- Toutes les pages font `require_once './includes/db.php'` — une seule source de vérité

Le trajet d'une requête, en une image

GET catalogue.php?categories[]=shonen&prix_max=10 : chaque étape, jusqu'au rendu final.

1

1 — Le navigateur

Le clic sur le filtre « Shonen » soumet le formulaire GET vers catalogue.php avec les paramètres choisis.

2

2 — PHP démarre

catalogue.php lit \$_GET['categories'] et \$_GET['prix_max'] avant tout affichage.

3

3 — Validation

Les slugs sont filtrés par expression régulière, le prix est borné entre 0 et 15.

4

4 — Construction du SQL

Le tableau \$where est assemblé dynamiquement selon les filtres réellement actifs.

5

5 — PDO

La requête préparée est exécutée avec des paramètres nommés — jamais de valeur concaténée.

6

6 — MySQL

Le moteur joint produits et categories, applique le filtre, retourne les lignes.

7

7 — Mise en forme

Chaque ligne devient une carte produit HTML, avec badge de stock calculé à la volée.

8

8 — Le navigateur affiche

Rendu entièrement côté serveur : la page complète arrive déjà prête, sans rechargement JS.

BLOC B

Le front-end

Maquetter, intégrer, puis rendre l'interface vivante avec JavaScript.

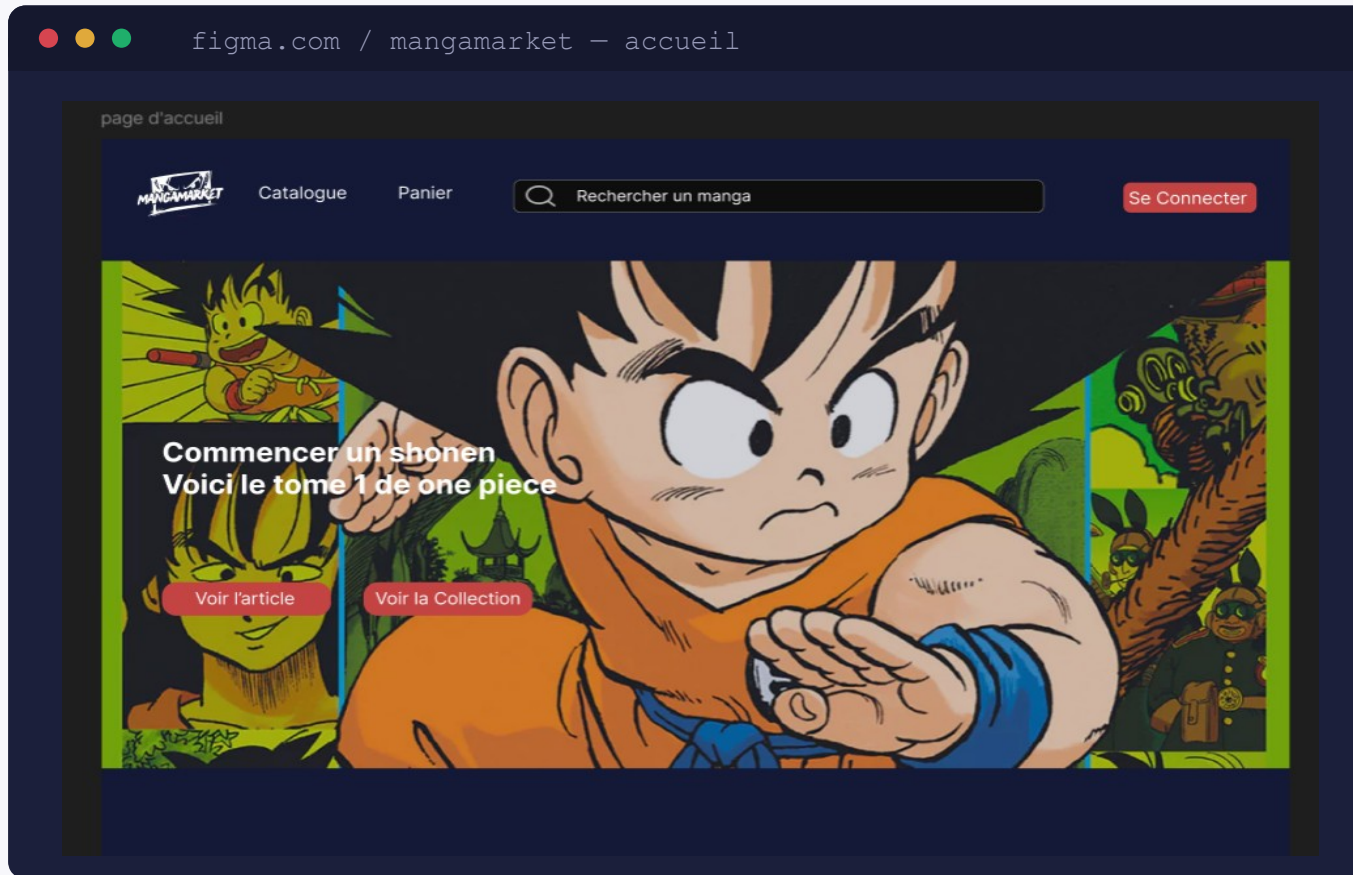
Maquettage

HTML / CSS

JavaScript

Maquetter les interfaces utilisateur

Neuf parcours-types dessinés avant la première ligne de code.



CE QUI A ÉTÉ MAQUETTÉ

- Accueil, catalogue, fiche produit, panier, tunnel de commande
- Connexion / inscription, profil client, back-office admin
- Composants réutilisables : cartes produit, badges de stock, formulaires
- Parcours complet du catalogue jusqu'à la confirmation de commande

Figma

Wireframe

UX/UI

Une charte graphique en dix variables

global.css centralise couleurs, rayons et ombres pour les 10 feuilles de style du site.

```
asset/css/global.css

1  :root {
2    --red:          #C0392B;
3    --gold-light:   #F4D03F;
4    --navy:         #0A0E1F;
5    --navy-light:   #1C2651;
6    --radius-md:    12px;
7    --shadow-glow:  0 0 20px rgba(192,57,43,.4);
8    --transition:   0.25s ease;
9  }
```

POURQUOI CENTRALISER

- Changer var(--red) une seule fois suffit à retexturer tous les boutons, badges et liens du site
- --shadow-glow crée l'effet de halo au survol, réutilisé sur .btn-primary et .card
- --transition uniformise la vitesse de toutes les animations (0,25 s, easing identique)
- Chaque feuille (catalogue.css, produit.css...) consomme ces variables sans les redéfinir

Réaliser les interfaces statiques

Intégration HTML depuis les maquettes, mise en forme CSS avec variables centralisées.

```
asset/css/catalogue.css

1  /* Sidebar de filtres, grille produits */
2  .filtre {
3    position: sticky; top: 90px;
4    background: var(--navy-light);
5  }
6  .fullcard {
7    display: grid;
8    grid-template-columns: repeat(auto-fill, minmax(210px, 1fr));
9  }
10 @media (max-width: 900px) {
11   .filtre { width: 100%; position: static; }
12 }
```

CE QUE ÇA PRODUIT

- Variables CSS centralisées (couleurs, rayons, ombres) pour une charte cohérente sur 10 feuilles de style
- Grille responsive qui s'adapte de 1 à 5 colonnes sans media query par carte
- Header fixe avec effet de flou (backdrop-filter), animations d'apparition en @keyframes
- Points de rupture mobile / tablette / desktop sur l'ensemble du site

Douze lignes pour une galerie d'images

produit.js : cliquer une miniature remplace l'image principale, sans rechargement.

```
asset/js/produit.js

1 document.querySelectorAll('.image-autre').forEach(thumb => {
2   thumb.addEventListener('click', () => {
3     const main = document.getElementById('main-img');
4     if (main) {
5       main.src = thumb.src;
6       document.querySelectorAll('.image-autre')
7         .forEach(t => t.classList.remove('active'));
8       thumb.classList.add('active');
9     }
10  });
11 });
```

UN SEUL ÉCOUTEUR, DÉLÉGUÉ À CHAQUE VIGNETTE

- `querySelectorAll().forEach()` attache un écouteur à chaque miniature au chargement
- `getElementById('main-img')` cible l'image principale une seule fois par clic
- La classe `.active` suit toujours la dernière miniature cliquée, retirée des autres
- Aucune requête réseau : tout se joue côté navigateur, instantanément

Développer la partie dynamique

Recherche instantanée, panier synchronisé, carrousel — sans framework.

```
asset/js/search.js

1  function rechercher(q) {
2    clearTimeout(debounceTimer);
3    if (q.trim().length < 2) { fermer(); return; }
4
5    debounceTimer = setTimeout(async() => {
6      const res  = await fetch(`${apiBase}?q=${q}`);
7      const data = await res.json();
8      afficherResultats(data, q.trim());
9    }, 200); // anti-rebond
10 }
```

CE QUI EST DYNAMIQUE

- Recherche en autocomplétion : debounce 200 ms, surlignage du terme, navigation au clavier
- Panier synchronisé entre BDD et localStorage, avec fusion automatique à la connexion
- Badge de quantité du panier mis à jour en temps réel après chaque action
- Carrousel de catégories (auto-play, swipe tactile) et menu burger mobile

Un clic « Ajouter », deux comportements possibles

header.js décide, à chaque clic, s'il parle à l'API ou au localStorage.

```
asset/js/header.js

1 document.addEventListener('click', async e => {
2   const btn = e.target.closest('.btn-add');
3   if (!btn || btn.disabled) return;
4
5   const data = await (await fetch(apiBase+'?action=list')).json();
6
7   if (data.connected) {
8     // POST action=add vers api/panier.php
9   } else {
10    // écrit directement dans localStorage['mm_cart']
11  }
12 });
```

LE CHOIX SE FAIT À CHAQUE CLIC

- Un seul écouteur délégué sur document capte tous les boutons .btn-add, même ceux ajoutés dynamiquement
- Une requête GET ?action=list vérifie l'état de connexion avant d'agir
- Si rupture ou stock atteint, le bouton affiche l'erreur puis se réinitialise après 1,5 à 2 s
- Dans tous les cas, updateCartBadge() rafraîchit le badge du header

La fusion du panier à la connexion

panier.js : le panier localStorage se vide au profit de la base, sans que le client ne s'en aperçoive.

```
asset/js/panier.js - loadCart()

1  async function loadCart() {
2    const data = await (await fetch(apiBase+'?action=list')).json();
3    if (data.connected) {
4      const local = getCartLocal();
5      if (local.length) {
6        for (const item of local)
7          await apiCall('add', { produit_id: item.id });
8        localStorage.removeItem('mm_cart');
9      }
10   } else renderItems(getCartLocal());
11 }
```

POURQUOI C'EST FIABLE

- Chaque article local est renvoyé à l'API add(), qui réapplique ses propres règles de stock
- Le localStorage n'est vidé qu'après confirmation de l'ajout en base, jamais avant
- Un client qui se connecte sur un autre appareil retrouve un panier déjà cohérent
- Aucune logique de fusion dupliquée : c'est toujours la même API qui décide

BLOC C

Le back-end & les données

La base relationnelle, l'accès aux données, et la logique métier qui protège le stock.

Modèle de données

Clés étrangères

Accès aux données

Transactions

Sept tables, une responsabilité chacune

Modélisation de la base relationnelle mangamarket.sql.

Table	Rôle
categories	Référentiel des genres de mangas (5 lignes de départ)
produits	Un enregistrement par tome : titre, auteur, prix, stock, image
utilisateurs	Comptes clients / admin — mot de passe haché (bcrypt)
commandes	En-tête de commande : livraison, total, statut
commande_lignes	Détail des articles d'une commande, prix figé à l'achat
panier	Panier persistant des clients connectés
messages_contact	Messages du formulaire de contact

Les six clés étrangères, et leur comportement

Ce que chaque contrainte produit concrètement au moment d'une suppression.

Table	Référence	ON DELETE	Effet concret
produits	categories	RESTRICT	Impossible de supprimer une catégorie qui contient des produits
commandes	utilisateurs	SET NULL	Compte supprimé → la commande reste, devenue anonyme
commande_lignes	commandes	CASCADE	Commande supprimée → ses lignes disparaissent avec elle
commande_lignes	produits	RESTRICT	Impossible de supprimer un produit déjà commandé
panier	utilisateurs	CASCADE	Compte supprimé → son panier est vidé automatiquement
panier	produits	CASCADE	Produit supprimé → il disparaît des paniers en cours

Un SQL qui s'assemble selon les filtres actifs

catalogue.php : la clause WHERE grandit uniquement si le client a réellement filtré.

```
views/catalogue.php

1 $where = ['p.prix <= :prix_max'];
2 $params = [':prix_max' => $prixMax];
3
4 if (!empty($catsActives)) {
5     $catsActives = array_filter($catsActives,
6         fn($s) => preg_match('/^[a-z_]+$/', $s));
7     $where[] = "c.slug IN ($placeholders)";
8     foreach ($catsActives as $i => $slug)
9         $params[":cat$i"] = $slug;
10 }
```

CE QUE ÇA EMPÊCHE

- preg_match() rejette tout slug qui ne ressemble pas à une catégorie avant qu'il n'atteigne le SQL
- Les placeholders nommés (:cat0, :cat1...) évitent tout conflit quand plusieurs catégories sont cochées
- \$prixMax est borné entre 0 et 15 avant même d'entrer dans \$params
- Résultat : la même requête sert 0, 1 ou 5 catégories cochées, sans jamais changer de forme

Développer les composants d'accès aux données

Une API JSON interne, consommée en Fetch par le panier et la recherche.

```
api/panier.php — case 'add'

1  $stock = (int)$stockStmt->fetchColumn();
2
3  if ($stock <= 0) {
4      echo json_encode(['error' => 'rupture']);
5  }
6  if ($qtyPanier + $quantite > $stock) {
7      echo json_encode(['error' => 'stock_insuffisant']);
8  }
9  $pdo->prepare("... ON DUPLICATE KEY UPDATE
    quantite = LEAST(quantite + VALUES(quantite), stock)"
);
```

PRINCIPES APPLIQUÉS

- Requêtes préparées PDO sur l'ensemble des accès — aucune valeur n'est jamais concaténée dans le SQL
- LEAST() empêche la quantité en panier de dépasser le stock, même en cas d'appels concurrents
- Réponses JSON uniformes : { connected, success, error? } — le front n'a qu'un seul contrat à gérer
- 3 endpoints internes : panier, produits, recherche

Un panier qui se corrige tout seul à l'affichage

api/panier.php — case 'list' : avant de répondre, l'API nettoie les incohérences.

```
api/panier.php — case 'list'

1 $pdo->prepare("UPDATE panier pa JOIN produits p,
2   ON p.id = pa.produit_id,
3   SET pa.quantite = p.stock,
4   WHERE pa.quantite > p.stock AND p.stock > 0")
5   ->execute([$userId]);
6
7 $pdo->prepare("DELETE pa FROM panier pa JOIN produits p ...
8   WHERE ... AND p.stock = 0")->execute([$userId]);
```

DEUX RÈGLES, AVANT MÊME D'AFFICHER

- Un article dont la quantité dépasse le stock (baissé entre-temps par l'admin) est automatiquement ramené au stock disponible
- Un article totalement en rupture est supprimé du panier avant que le client ne le voie
- Le client n'a jamais besoin de rafraîchir : chaque appel à ?action=list ré-applique ces règles

La recherche : dix lignes, deux colonnes interrogées

api/search_api.php répond à search.js à chaque frappe (après anti-rebond).

```
api/search_api.php

1  $like = '%' . $q . '%';
2
3  $stmt = $pdo->prepare("SELECT p.id, p.titre, p.tome, p.prix,
4      p.image, c.slug AS categorie
5      FROM produits p JOIN categories c ON c.id = p.categorie_id
6      WHERE p.titre LIKE :q1 OR c.nom LIKE :q2
7      ORDER BY p.titre LIMIT 10");
8
9  $stmt->execute([':q1' => $like, ':q2' => $like]);
```

CE QUI EST DÉLIBÉRÉ

- Deux paramètres nommés distincts (:q1, :q2) pour la même valeur
 - PDO l'exige quand un paramètre sert deux fois
- La recherche porte aussi bien sur le titre du manga que sur le nom de la catégorie
- LIMIT 10 borne la charge, quelle que soit la popularité du terme recherché
- mb_strlen(\$q) < 1 côté API court-circuite les requêtes vides avant même la connexion

Le formulaire de contact, validé avant d'écrire

contact.php : rien n'entre en base sans avoir passé cinq contrôles.

```
views/contact.php

1  if (empty($nom))    $errors[] = "Le nom est requis.";
2  if (empty($prenom)) $errors[] = "Le prénom est requis.";
3  if (!filter_var($mail, FILTER_VALIDATE_EMAIL))
4      $errors[] = "L'adresse email est invalide.";
5
6  if (empty($errors)) {
7      $pdo->prepare("INSERT INTO messages_contact
8          (nom,prenom,telephone,email,message) VALUES (?, ?, ?, ?, ?)")
9      ->execute([$nom,$prenom,$tel,$mail,$message]);
10 }
```

RIEN N'EST FAIT CONFIANCE

- Chaque champ obligatoire est vérifié individuellement, avec un message d'erreur dédié
- FILTER_VALIDATE_EMAIL rejette un format invalide avant toute tentative d'enregistrement
- L'INSERT ne s'exécute que si le tableau \$errors est resté vide
- Les messages sont ensuite consultables et marquables « lus » depuis le back-office

Connexion et inscription : jamais de mot de passe en clair

creecompte.php gère les deux formulaires, avec le même souci de sécurité.

```
views/creecompte.php

1 // Connexion
2 $user = $stmt->fetch();
3 if ($user && password_verify($password, $user['password'])) {
4     $_SESSION['user_id'] = $user['id'];
5     $_SESSION['user_role'] = $user['role'];
6 }
7
8 // Inscription
9 $hash = password_hash($password, PASSWORD_BCRYPT);
10 $pdo->prepare("INSERT INTO utilisateurs (...) VALUES (?, ?, ?, ?)")
11     ->execute([$prenom, $nom, $email, $hash]);
```

CE QUE ÇA GARANTIT

- password_hash() en BCRYPT génère un sel unique à chaque inscription : deux mots de passe identiques donnent deux hachages différents
- password_verify() compare sans jamais déchiffrer — le mot de passe original n'est jamais reconstitué
- Le rôle est toujours forcé à 'client' à l'inscription : personne ne peut s'auto-promouvoir admin
- \$_SESSION['user_role'] pilote ensuite tout l'affichage du header

Deux gardiens, quatre lignes chacun

auth_client.php et auth_admin.php sont inclus en toute première ligne des pages protégées.

```
includes/auth_admin.php

1  if (session_status() === PHP_SESSION_NONE) session_start();
2
3  if (empty($_SESSION['user_id'])) {
4      header('Location: ../../views/creecompte.php');
5      exit;
6  }
7  if (($_SESSION['user_role'] ?? 'client') !== 'admin') {
8      header('Location: ../../index.php?error=acces_refuse');
9      exit;
10 }
```

INCLUS AVANT TOUT LE RESTE

- require_once '../includes/auth_admin.php' est la toute première instruction de chaque page admin
- Deux redirections distinctes : non connecté → connexion ; connecté mais pas admin → accueil avec message
- auth_client.php applique la même logique, plus légère, pour les pages profil et commandes
- Aucune page protégée ne contient sa propre logique de contrôle : le comportement est garanti identique partout

Développer les composants métier

Le tunnel de commande : une transaction SQL qui ne laisse aucune place à l'incohérence.

```
views/achat.php - validation de commande

1  $pdo->beginTransaction();
2  $stmt = $pdo->prepare("SELECT prix, stock FROM produits
    WHERE id = ? FOR UPDATE"
);
3
4  if (!$prod || $prod['stock'] <= 0) {
5      $pdo->rollBack(); // aucune trace laissée
6  } else {
7      $pdo->prepare("INSERT INTO commandes ...")->execute(...);
8      $pdo->prepare("UPDATE produits SET stock = stock - 1 ...");
9      $pdo->commit();
10 }
```

CE QUE ÇA GARANTIT

- Authentification bcrypt (password_hash / password_verify), sessions PHP contrôlées par rôle
- Fichiers auth_client.php / auth_admin.php réutilisés en première ligne des pages protégées
- Toute la séquence commande + lignes + décrémentation de stock réussit ensemble, ou échoue ensemble
- En cas d'erreur, rollBack() annule tout : jamais de commande à moitié enregistrée

FOR UPDATE : deux clients, le dernier exemplaire

Le même scénario joué deux fois. Stock initial : 1 exemplaire. Deux clients valident au même instant.

t	Client A	Client B	Sans FOR UPDATE	Avec FOR UPDATE
1	BEGIN	—	stock = 1	stock = 1
2	SELECT stock	—	lit 1	lit 1, verrouille la ligne
3	—	BEGIN	—	—
4	—	SELECT stock	lit 1 aussi	attend la fin de A
5	stock > 0 ✓	—	test passé	test passé
6	UPDATE stock-1	stock > 0 ✓	test passé aussi	(en attente)
7	COMMIT	UPDATE stock-1	stock = 0	stock = 0
8	—	COMMIT	deux ventes, un livre	verrou levé, B relit stock = 0 → ROLLBACK

Sans verrou, deux clients peuvent tous deux valider le dernier exemplaire : c'est exactement le scénario que FOR UPDATE empêche dans achat.php.

Le back-office : cinq lignes pour ajuster un stock

admin/produits.php : chaque ligne du tableau produit a son propre mini-formulaire.

views/admin/produits.php

```
1 if ($_SERVER['REQUEST_METHOD'] === 'POST'
2     && ($_POST['action'] ?? '') === 'update_stock') {
3     $id      = (int)$_POST['produit_id'];
4     $stock = max(0, (int)$_POST['stock']);
5     $pdo->prepare("UPDATE produits SET stock = ? WHERE id = ?")
6     ->execute([$stock, $id]);
7     header('Location: produits.php?updated=1'); exit;
8 }
```

SIMPLE, MAIS PAS NAÏF

- `max(0, ...)` empêche un stock négatif même si l'admin saisit une valeur absurde
- Chaque ligne du tableau produit son propre <form>, avec l'id du produit en champ caché
- La redirection ?updated=1 après le POST évite la resoumission du formulaire au rafraîchissement
- Le même schéma sert à `update_prix` et à supprimer, avec juste l'action qui change

Garder deux tomes du même manga cohérents

admin/produits.php : avant d'ajouter un tome, on vérifie si le manga existe déjà.

```
views/admin/produits.php — action 'ajouter'

1 $ref = $pdo->prepare("SELECT titre, auteur, categorie_id
2   FROM produits WHERE titre = ? LIMIT 1");
3 $ref->execute([$titre]);
4 $existant = $ref->fetch();
5
6 if ($existant) {
7     $titre      = $existant['titre'];
8     $auteur     = $existant['auteur'];
9     $categorieId = (int)$existant['categorie_id'];
10 }
```

UN VRAI BUG, RÉELLEMENT CORRIGÉ

- Constat initial : « Dragon Ball » (tome 35) et « dragon ball » (tome 1) coexistaient en base avec des casses différentes
- Avant chaque insertion, on cherche un produit du même titre déjà existant
- S'il existe, son orthographe exacte de titre, auteur et catégorie remplace ce que l'admin a tapé
- Chaque tome d'un même manga reste ainsi strictement identique sur ces trois champs

BLOC D

Sécurité & démonstration

Les mesures mises en place, et le parcours complet d'un achat.

Sécurité

Parcours utilisateur

Six mesures, six risques couverts

Chaque mesure répond à un risque précis, pas à une checklist générique.

Risque	Mesure appliquée
Injection SQL	Requêtes préparées PDO sur l'intégralité des accès à la base
Faible XSS	htmlspecialchars() systématique sur toute sortie HTML générée depuis la BDD
Mot de passe compromis	Hachage bcrypt (password_hash / password_verify), jamais de clair
Accès non autorisé	Contrôle de session + rôle avant chaque page profil / admin
Survente en cas de concurrence	Verrouillage FOR UPDATE + transaction avant décrémentation du stock
Données invalides	Validation serveur systématique sur tous les formulaires (contact, compte, commande)

Changer son mot de passe, en trois vérifications

profil.php : on ne fait confiance ni au formulaire, ni à la session.

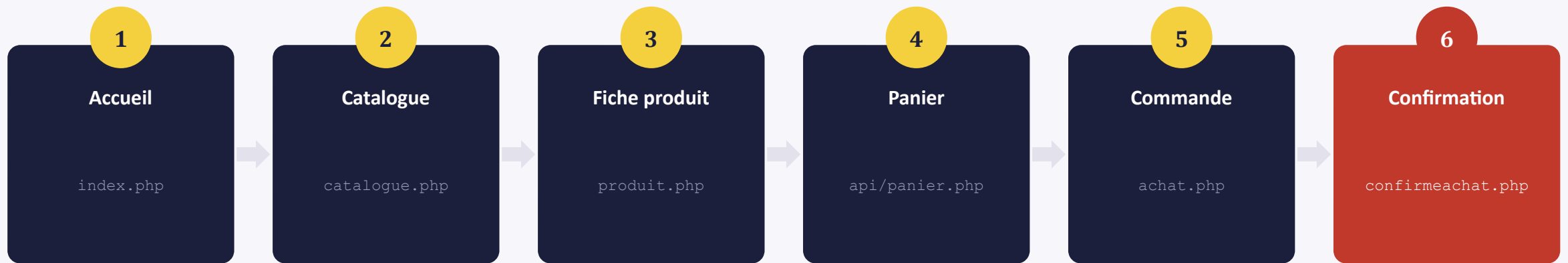
```
views/profil.php

1  if (strlen($new) < 8)
2      $pwErrors[] = "8 caractères minimum.";
3  if ($new !== $confirm)
4      $pwErrors[] = "Les mots de passe ne correspondent pas.";
5
6  if (empty($pwErrors)) {
7      if (!password_verify($current, $hash)) {
8          $pwErrors[] = "Mot de passe actuel incorrect.";
9      } else {
10         $pdo->prepare("UPDATE utilisateurs SET password = ? WHERE id = ?")
11         ->execute([password_hash($new, PASSWORD_BCRYPT), $userId]);
12     }
13 }
```

TROIS VÉRIFICATIONS AVANT D'ÉCRIRE

- Longueur minimale (8 caractères) et confirmation identique, comme à l'inscription
- Le mot de passe actuel doit être revérifié par `password_verify()` avant tout changement — même pour un compte déjà connecté
- Le nouveau mot de passe est immédiatement rehaché en `bcrypt`, jamais recopié tel quel
- Aucune des trois étapes n'est sautée, même si les précédentes ont réussi

Le parcours d'achat, de l'accueil à la commande



Chaque flèche correspond à une vérification serveur (stock, session, validité des données) — impossible de commander une rupture.

La dernière étape : confirmer, puis nettoyer

confirmeachat.php affiche le succès ; confirmeachat.js efface la trace côté client.

```
confirmeachat.php + confirmeachat.js

1 // confirmeachat.php
2 <h1>Merci pour votre commande !</h1>
3 <p>Vous recevrez un email de confirmation...</p>
4
5 // confirmeachat.js — exécuté au chargement
6 localStorage.removeItem('mm_cart');
```

POURQUOI DEUX FICHIERS

- Le panier BDD est déjà vidé côté serveur, dans achat.php, avant même la redirection
- Le panier localStorage (invités) ne peut être vidé que côté navigateur : d'où ce petit script dédié
- removeItem() ne provoque aucune erreur si la clé n'existe déjà plus (ex. client connecté)
- Résultat : quel que soit le parcours suivi, le panier repart toujours à zéro après une commande

BLOC E

Bilan

Compétences mobilisées, difficultés rencontrées, et perspectives.

Compétences

Difficultés

Perspectives

Compétences mobilisées

Les deux activités-types du titre `DWWM`, illustrées par le projet.

Activité-type	Compétence	Illustration dans MangaMarket
Front-end	Maquetter des interfaces	Maquettes Figma de toutes les pages du site
Front-end	Réaliser des interfaces statiques	Intégration HTML/CSS responsive, charte à variables
Front-end	Développer la partie dynamique	Recherche, panier et carrousel en JavaScript natif
Back-end	Mettre en place une BDD relationnelle	7 tables, 6 clés étrangères, contraintes d'intégrité
Back-end	Développer l'accès aux données	API REST interne en PHP / PDO
Back-end	Développer les composants métier	Authentification, transactions, back-office admin

Difficultés rencontrées & solutions apportées

Cohérence entre les tomes

PROBLÈME

Un même manga pouvait être saisi avec des variantes de titre ou de catégorie selon le tome.

SOLUTION

Alignement automatique du titre / auteur / catégorie sur la fiche existante lors de l'ajout d'un nouveau tome.

Panier invité vs connecté

PROBLÈME

Le panier devait fonctionner sans compte, puis retrouver son contenu après connexion.

SOLUTION

Panier localStorage pour les invités, fusion automatique et silencieuse vers la BDD à la connexion.

Concurrence sur le stock

PROBLÈME

Deux clients pouvaient valider le dernier exemplaire au même instant.

SOLUTION

Transaction SQL avec verrouillage FOR UPDATE avant toute décrémentation (voir Bloc C).

M E R C I

Des questions sur un point en particulier ?

Architecture, base de données, back-office ou front-end : chaque partie de ce diaporama s'appuie sur du code réellement écrit pour MangaMarket.

Matteo Bils — Dossier professionnel DWWM